GIT

Complete Developer Study Notes

Concepts • Practical Examples • Projects • Interview Preparation

Prepared by: Rahul Yadav

20 Pages

Rahul Yadav

1. Introduction to Git

1.1 What is Git ?

- Git is a distributed version control system used to track changes in source code during software development.
- It helps multiple developers work on the same project efficiently.
- It keeps a history of all changes, so we can go back to any previous version if needed.
- Git is widely used in companies for collaboration, code management and release management.

1.2 Key Features of Git

- **Distributed** - Every developer has a full copy of the repository.
- **Version control** - Tracks all changes in files.
- **Branching and merging** - Allows parallel development.
- **Fast and lightweight** - Works efficiently even with large projects.
- **Supports offline work** - Most operations can be done without internet.
- **Data integrity** - Uses SHA-1 hashing to ensure code is not corrupted.

1.3 Git vs Other Version Control Systems

Feature	Git	SVN (Subversion)	Mercurial
Architecture	Distributed	Centralized	Distributed
Local repository	Yes (full copy)	No	Yes
Speed	Fast	Slower	Fast
Branching	Easy and lightweight	Complex	Moderate
Offline work	Yes	Limited	Yes

1.4 Git Architecture



- Working Directory - your project files.
- Staging Area (Index) - files added for the next commit.
- Repository - stores the complete history of commits.

1.5 Common Use Cases

- Track changes in source code.
- Collaborate with multiple developers.
- Maintain different versions of the project.
- Experiment with new features using branches.
- Revert to previous versions if something goes wrong.

Rahul Yadav

2. Installation and Setup

2.1 How to Install Git ?

- Download from official website: https://git-scm.com/
- Choose your OS (Windows, macOS, Linux) and follow the installer.
- Verify installation using:

```
git --version          # Should show Git version
```

2.2 Initial Configuration

- Set your name and email (used in commits):

```
git config --global user.name "Rahul Yadav"
git config --global user.email "rahul@example.com"
```

- Check configuration:

```
git config --list          # View all config settings
```

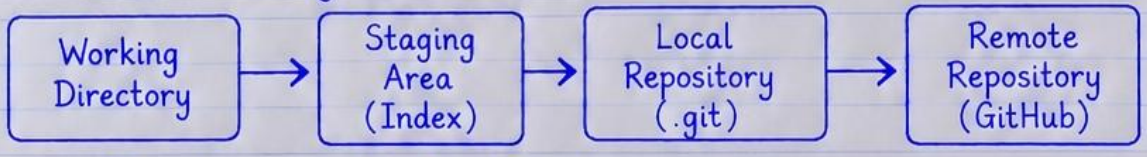
2.3 Basic Git Commands

Command	Description
git init	Initialize a new Git repository
git clone <url>	Clone a remote repository
git status	Check working directory status
git add <file>	Stage changes for next commit
git commit -m "msg"	Save staged changes with message
git push	Push commits to remote repository
git pull	Fetch and merge changes from remote

2.4 Example: Initialize a Repository

```
$ mkdir my-project
$ cd my-project                # Creates a .git folder
$ git init
```

2.5 Understanding Git Workflow



- ✓ Make changes in Working Directory
- ✓ Add changes to Staging Area
- ✓ Commit changes to Local Repository
- ✓ Push to Remote Repository

Rahul Yadav

3. Basic Git Commands

3.1 Checking Repository Status

- We can check the current status of our working directory using:

```
git status
```

- It shows which files are modified, staged or untracked.
- It helps us understand what changes will be included in the next commit.

3.2 Adding Files to Staging Area

- We use git add command to add changes to the staging area.
- To add a single file: `git add file_name`
- To add all files: `git add .`

3.3 Committing Changes

- After adding files, we commit the changes.

```
git commit -m "Your commit message"
```

- Commit message should be short and meaningful.

3.4 Viewing Commit History

- Use the following command to see commit history:

```
git log
```

- For a short and clean format:

```
git log --oneline
```

Useful git log options:

- --oneline (short format)
- --graph (show branch graph)
- --all (show all branches)
- --decorate (show branch names)

3.5 Common Git Commands

Command	Description
git init	Initialize a new Git repository
git status	Check the status of changes
git add <file>	Add a file to staging area
git add .	Add all files to staging area
git commit -m "message"	Commit the changes with a message
git log	View commit history
git log --oneline	View short commit history
git diff	Show changes between working directory and staging area
git show <commit_id>	Show details of a specific commit
git rm <file>	Remove a file from working directory and staging area
git mv <old> <new>	Rename or move a file

3.6 Practical Example

1. Create a new folder and go inside it.
2. Run git init
3. Create a file (e.g., app.java) and add some code.
4. Run git add .
5. Run git commit -m "Initial commit"
6. Run git log --oneline to see the commit.

Rahul Yadav

4. Git Branching and Merging

4.1 What is Branching ?

- Branching allows us to work on new features, bug fixes or experiments without affecting the main codebase.
- Each branch is an independent line of development.
- It helps multiple developers work on different features in parallel.

4.2 Creating and Switching Branches

- Create a new branch:

```
git branch <branch-name>
```

- Switch to a branch:

```
git checkout <branch-name>
```

- Create and switch to a new branch (short command):

```
git checkout -b <branch-name>
```

List all branches:

Local branches

```
git branch
```

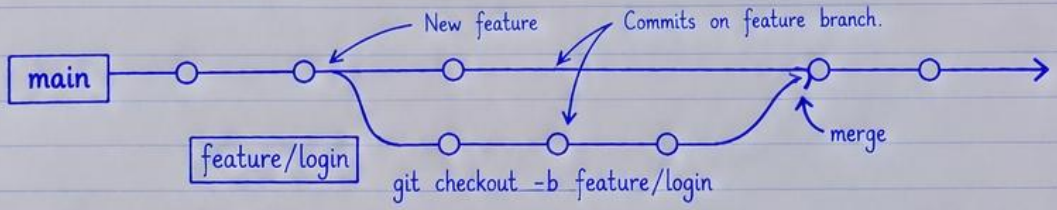
Remote branches

```
git branch -r
```

All branches

```
git branch -a
```

4.3 Branching Workflow



4.4 Merging Branches

- After completing the work, we can merge the feature branch into main.
- First switch to the target branch (usually main).
- Then run the merge command.

```
git checkout main
git merge <branch-name>
```

Types of Merge:

- Fast Forward Merge (simple)
- Three-way Merge (default)
- Squash Merge (single commit)
- Rebase and Merge (linear history)

4.5 Rebase (Alternative to Merge)

- Rebase moves the commits of one branch on top of another branch.
- It keeps the commit history clean and linear.

```
git checkout <branch-name>
git rebase main
```

When to use Rebase?

- To maintain a clean linear history.
- In personal or small teams.
- Avoid rebasing shared branches.

4.6 Deleting Branches

- Delete a local branch: `git branch -d <branch-name>`
- Force delete (if not merged): `git branch -D <branch-name>`
- Delete a remote branch: `git push origin --delete <branch-name>`

4.7 Best Practices

- Use meaningful branch names (e.g., feature/login, bugfix/payment-issue).
- Keep branches small and focused.
- Regularly sync with the latest main branch.
- Delete merged branches to keep the repository clean.

Rahul Yadav

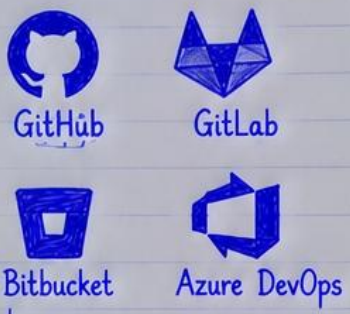
5. Git Remote Repository

5.1 What is Remote Repository ?

- A remote repository is a version of your project that is hosted on the internet (e.g., GitHub, GitLab, Bitbucket).
- It allows multiple developers to collaborate on the same project.
- We can push our local changes to remote and pull changes from remote.
- It acts as a central location to store the project code and history.

5.2 Popular Git Hosting Platforms

- GitHub – Most popular, widely used
- GitLab – Self-hosted option, CI/CD features
- Bitbucket – Popular in enterprise, good with Jira
- Azure DevOps – Used in Microsoft ecosystem



5.3 Adding a Remote Repository

- We can add a remote repository using the following command:

```
git remote add origin <repository-url>
```

- Example:

```
git remote add origin https://github.com/username/project.git
```

Note:
- origin is a common name for the remote.
- We can use any name instead of origin.

5.4 Viewing Remote Repositories

- To list all remote repositories:

```
git remote -v
```

- To see details of a specific remote:

```
git remote show origin
```

Sample Output (git remote -v):

```
origin  https://github.com/username/project.git (fetch)
origin  https://github.com/username/project.git (push)
```

5.5 Pushing Changes to Remote

- To push local commits to remote repository:

```
git push origin <branch-name>
```

- Example:

```
git push origin main
```

First Time Push (if branch not exists):

```
git push -u origin main
```

- -u sets the upstream branch.
- Next time we can simply use:

```
git push
```

5.6 Pulling Changes from Remote

- To get the latest changes from remote repository:

```
git pull origin <branch-name>
```

- Example:

```
git pull origin main
```

Pull = Fetch + Merge
- It downloads the latest changes (fetch) and merges them with our local branch.
- If there are conflicts, we need to resolve them manually.

5.7 Best Practices

- Always pull the latest changes before pushing.
- Use meaningful commit messages.
- Work on separate branches for new features.
- Avoid pushing directly to main branch (use Pull Requests).
- Keep your remote repository secure (use SSH or access tokens).

Rahul Yadav

6. Working with Remote Repositories

6.1 What is a Remote Repository ?

- A remote repository is a version of our project that is hosted on a server (e.g., GitHub, GitLab, Bitbucket) and can be accessed over the internet.
- It allows multiple developers to collaborate on the same project.
- We can push our local changes to remote and pull changes from remote.
- It keeps our code safe and provides backup.

6.2 Connecting Local Repository to Remote

- To connect our local repository to a remote repository, we use:

```
git remote add origin <repository-url>
```

- Example (GitHub):

```
git remote add origin https://github.com/username/project.git
```

6.3 Remote Repository Workflow



6.4 Common Remote Commands

Command	Description
git remote -v	List remote repositories
git remote show origin	Show details of remote repository
git push origin <branch-name>	Push local changes to remote
git pull origin <branch-name>	Pull latest changes from remote
git fetch origin	Fetch latest changes without merging
git clone <repository-url>	Create a local copy of a remote repository

6.5 Cloning a Remote Repository

- To get a copy of an existing remote repository, we use:

```
git clone <repository-url>
```

- Example:

```
git clone https://github.com/username/project.git
```

- This creates a new folder with the project name and downloads all files.

6.6 Working with Different Branches on Remote

- Push a new branch to remote:

```
git push -u origin <branch-name>
```

- Pull a specific branch:

```
git pull origin <branch-name>
```

- Delete a remote branch:

```
git push origin --delete <branch-name>
```

6.7 Best Practices

- Always pull latest changes before pushing.
- Use meaningful branch names.
- Avoid pushing large files.
- Use .gitignore to exclude unnecessary files.
- Keep your remote repository secure (use SSH or Personal Access Token).

Rahul Yadav

7. Git Stash (Temporary Storage)

7.1 What is Git Stash ?

- Git stash allows us to temporarily save our uncommitted changes.
- It is useful when we want to switch branches or work on something else without committing the current changes.
- It keeps our working directory clean and safe.

7.2 Common Stash Commands

Command	Description
git stash	Stash uncommitted changes
git stash -m "message"	Stash with a message
git stash list	List all stashed changes
git stash show	Show details of the latest stash
git stash apply	Apply the latest stash (keep in stash list)
git stash pop	Apply the latest stash and remove from list
git stash drop	Remove a specific stash
git stash clear	Remove all stashes

7.3 Stash Workflow



7.4 Practical Example

- Make some changes in a file (not committed).
- Stash the changes:

```
git stash -m "WIP: payment module changes"
```

- View stash list: `git stash list`
- Apply the latest stash: `git stash apply`
- Apply and remove from list: `git stash pop`

7.5 Example Output

```
$ git stash list
stash@{0}: WIP: payment module changes
stash@{1}: WIP: login bug fix
stash@{2}: On main: UI changes
```

```
$ git stash show
 payment.txt | 5 +++--
 1 file changed, 3 insertions(+),
 2 deletions(-)
```

7.6 Best Practices

- Use meaningful stash messages.
- Use stash for temporary work, not long-term storage.
- Keep stash list clean by removing old stashes.
- Use git stash -u to also stash untracked files.
- Prefer commit instead of stash for long-term work.

Rahul Yadav

8. Git Configuration

8.1 What is Git Configuration ?

- Git configuration is used to set user details, default behavior and preferences.
- It can be set at different levels (system, global, local).
- These settings are stored in configuration files.
- It helps in maintaining consistent identity and behavior across projects.

8.2 Types of Git Configuration

Type	Command Option	Scope / Description
System	--system	Applies to all users on the system.
Global	--global	Applies to the current user (all repositories).
Local	--local	Applies only to the current repository.

8.3 Setting User Name and Email

- Set your name (global):

```
git config --global user.name "Rahul Yadav"
```

- Set your email (global):

```
git config --global user.email "rahul@example.com"
```

- Verify the configuration:

```
git config --list
```

Note:
- Use the same email which you use in GitHub.
- If you use different email, it will be considered as a different contributor.

8.4 Checking Configuration Values

- To check a specific configuration: `git config user.name`
- To check all configuration (global): `git config --global --list`
- To check system configuration: `git config --system --list`
- To check local repository configuration: `git config --local --list`

8.5 Git Configuration Files

Level	File Location	Description
System	/etc/gitconfig	System-wide configuration.
Global	~/.gitconfig	User-specific configuration.
Local	.git/config (inside repository)	Repository-specific configuration.

8.6 Useful Configuration Settings

Command	Description
git config --global core.editor "code --wait"	Set default editor (VS Code)
git config --global init.defaultBranch main	Set default branch name to main
git config --global core.autocrlf true	Handle line ending (Windows)
git config --global color.ui true	Enable colored output in terminal
git config --global pull.rebase false	Set default pull behavior

8.7 Best Practices

- Always set your correct name and email.
- Use meaningful editor and default branch name.
- Keep global configuration consistent across all your machines.
- Verify configuration using git config --list.
- Use repository-specific configuration when needed.

Rahul Yadav

9. Git Ignore (.gitignore)

9.1 What is .gitignore ?

- .gitignore is a special file that tells Git which files and folders to ignore.
- It prevents unwanted files from being tracked in the repository.
- Useful for ignoring build files, temporary files, IDE files, environment files, etc.
- Helps to keep the repository clean and lightweight.

9.2 Where to Create .gitignore ?

- Create a file named .gitignore in the root directory of your project.
- It can also be created inside subdirectories to ignore files in specific folders.

Example .gitignore Structure:

```
# Ignore build files
/build/
# Ignore IDE files
.idea/
# Ignore environment files
.env
# Ignore log files
*.log
```

9.3 Basic Syntax and Rules

- Each pattern is written on a new line.
- Lines starting with # are comments.
- Git uses pattern matching to ignore files and folders.
- You can use wildcards (*), directory (/) and negation (!).

9.4 Common .gitignore Patterns

Pattern	Description	Example
*.log	Ignore all log files	app.log, error.log
*.tmp	Ignore temporary files	temp.tmp
/build/	Ignore build directory	build/ folder
.idea/	Ignore JetBrains IDE files	.idea/ folder
*.class	Ignore compiled Java files	Test.class
node_modules/	Ignore Node.js dependencies	node_modules/ folder
.env	Ignore environment file	.env
*.iml	Ignore IntelliJ module files	project.iml
dist/	Ignore distribution folder	dist/ folder

9.5 Examples

Example 1: Ignore specific files and folders

```
# Ignore all log files
*.log
# Ignore build directory
/build/
# Ignore IDE files
.idea/
# Ignore environment file
.env
```

Example 2: Ignore with wildcards and negation

```
# Ignore all .txt files
*.txt
# Ignore all files in temp folder
temp/*
# Ignore everything in a folder except a specific file
docs/*
!docs/readme.txt   # This file will be tracked
```

9.6 Checking Ignored Files

- To check if a file is ignored, use:

```
git check-ignore -v <file-name>
```

- Example:

```
git check-ignore -v .env
```

→ Shows which rule is ignoring the file and from which .gitignore file.

9.7 Best Practices

- Always use .gitignore to exclude unnecessary files.
- Include .gitignore in your repository.
- Use comments to explain each rule.
- Keep it updated as the project grows.
- Avoid ignoring important files.
- Use templates for your technology (e.g., Java, Python, Node.js).

Rahul Yadav

10. Git Diff (Compare Changes)

10.1 What is Git Diff ?

- Git diff is used to see the differences between files, commits, branches, etc.
- It shows what changes we have made but not yet committed.
- It helps to review changes before committing.
- Useful for debugging, code reviews and understanding modifications.

10.2 Common Git Diff Commands

Command	Description
git diff	Show changes in working directory (not staged).
git diff --staged	Show changes that are staged (ready to commit).
git diff <file-name>	Show changes for a specific file.
git diff HEAD	Show all changes compared to the last commit.
git diff <commit1> <commit2>	Show differences between two commits.
git diff <branch1> <branch2>	Show differences between two branches.

10.3 Git Diff Workflow



10.4 Practical Examples

Example 1: See all changes (not staged)

```
git diff
```

Example 2: See staged changes

```
git diff --staged
```

Example 3: See changes in a specific file

```
git diff app.js
```

Example 4: Compare two commits

```
git diff abc123 def456
```

Example 5: Compare two branches

```
git diff main feature-branch
```

Example 6: Show only file names with changes

```
git diff --name-only
```

10.5 Sample Output

```
diff --git a/app.js b/app.js
index e69de29..4b825dc 100644
--- a/app.js
+++ b/app.js
@@ -1,3 +1,4 @@
- console.log("Hello World");
+ console.log("Hello Git Diff");
  console.log("End");
```

Explanation:

- Lines with - are removed.
- Lines with + are added.
- It also shows the file name, index and line numbers.

10.6 Best Practices

- Always review changes using git diff before committing.
- Use git diff --staged to double-check staged files.
- Use git diff <file-name> for specific file review.
- Compare branches or commits to understand the impact.
- Make it a habit to review your changes for clean and bug-free code.

Rahul Yadav

11. Git Branching

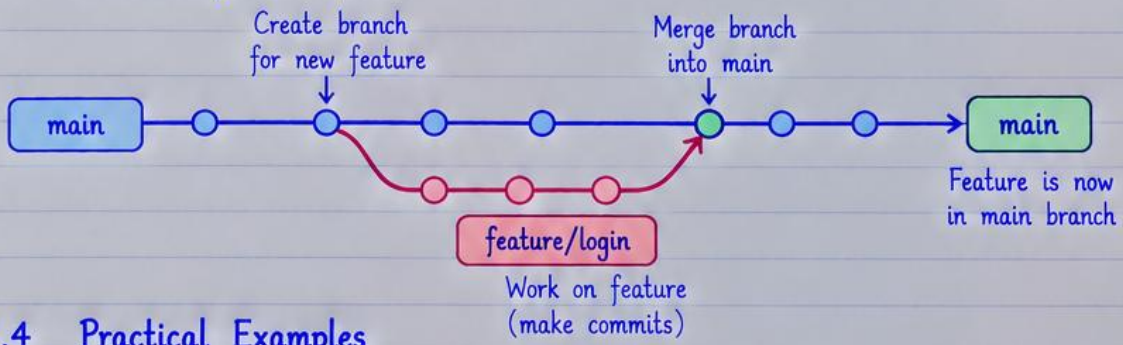
11.1 What is a Git Branch ?

- A branch is a separate line of development in Git.
- It allows us to work on new features, bug fixes or experiments without affecting the main codebase.
- Each branch has its own history, and later it can be merged into another branch (e.g., main).
- Branches help multiple developers work in parallel.

11.2 Common Branching Commands

Command	Description
git branch	List all local branches
git branch -a	List all local and remote branches
git branch <branch-name>	Create a new branch
git checkout <branch-name>	Switch to an existing branch
git checkout -b <branch-name>	Create and switch to a new branch
git branch -d <branch-name>	Delete a branch (after merge)
git branch -D <branch-name>	Force delete a branch (not recommended)
git push -u origin <branch-name>	Push a new branch to remote
git branch -m <old-name> <new-name>	Rename a branch

11.3 Branching Workflow



11.4 Practical Examples

Example 1: Create and switch to a new branch

```
git checkout -b feature/login
```

Example 2: List all branches

```
git branch -a
```

Example 3: Push branch to remote

```
git push -u origin feature/login
```

Example 4: Merge branch into main

```
git checkout main
git merge feature/login
```

Example 5: Delete a branch

```
git branch -d feature/login
```

Example 6: Rename a branch

```
git branch -m old-branch new-branch
```

11.5 Best Practices

- Use meaningful branch names (e.g., feature/login, bugfix/payment-issue).
- Keep branches small and focused.
- Pull latest changes from main before merging.
- Delete branches after merging to keep the repository clean.
- Use a consistent branching strategy (e.g., Git Flow or GitHub Flow).
- Avoid making direct changes to the main branch.

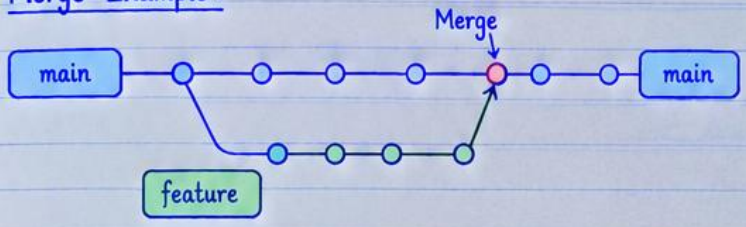
Rahul Yadav

12. Git Merge & Rebase

12.1 What is Git Merge ?

- Git merge is used to combine changes from one branch into another branch.
- It creates a new commit (merge commit) and keeps the history of both branches.
- It is safe and does not rewrite history.

Merge Example :



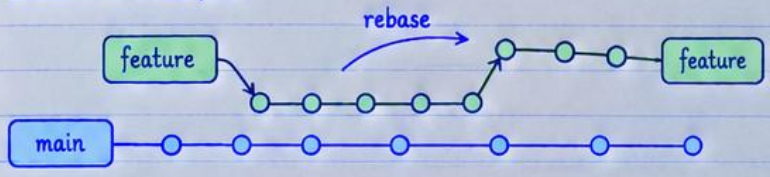
Example Commands :

```
git checkout main
git merge feature
# or
git merge --no-ff feature
    (create merge commit)
```

12.2 What is Git Rebase ?

- Git rebase is used to move or apply commits from one branch on top of another branch.
- It keeps a linear history (no merge commits).
- It rewrites commit history (changes commit IDs).

Rebase Example :



Example Commands :

```
git checkout feature
git rebase main
# or
git rebase main
```

12.3 Key Differences

Feature	Git Merge	Git Rebase
History	Keeps both histories	Linear history (no merge commit)
Commit IDs	Creates new merge commit	Rewrites commit IDs
Use Case	Good for shared branches	Good for personal branches
Command	git merge	git rebase
Complexity	Simple	Slightly complex
Risk	Low	Higher (rewrites history)

12.4 When to Use What ?

- Use Merge when working with shared branches (e.g. team collaboration).
- Use Rebase when you want a clean, linear history (e.g. before pushing to remote or for feature branches).
- Avoid rebasing on shared/public branches (it can confuse others).

12.5 Best Practices

- Always pull latest changes before merging or rebasing.
- Use rebase for local feature branches only.
- Avoid force push on shared branches (use --force-with-lease if needed).
- Keep commit history clean and meaningful.
- Communicate with your team before rebasing shared branches.

Rahul Yadav

13. Git Remote & Collaboration

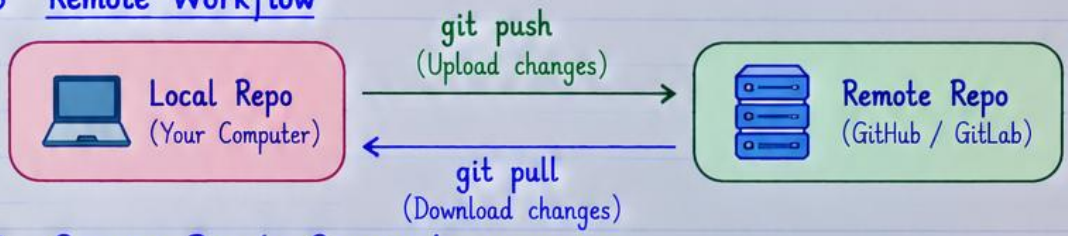
13.1 What is a Remote Repository ?

- A remote repository is a version of your project hosted on a server (e.g. GitHub, GitLab, Bitbucket).
- It allows multiple developers to collaborate on the same project.
- You can push your local changes to the remote and pull changes done by others.

13.2 Popular Git Hosting Platforms



13.3 Remote Workflow



13.4 Common Remote Commands

Command	Description
`git remote -v`	List all remote repositories
`git remote add origin <url>`	Add a new remote repository
`git push -u origin main`	Push local branch to remote (and set upstream)
`git pull origin main`	Pull latest changes from remote
`git fetch origin`	Download latest changes (without merging)
`git remote remove origin`	Remove a remote repository
`git remote set-url origin <url>`	Change remote repository URL

13.5 Example: Pushing Code to GitHub

```
git init                                                  # Initialize local repository
git remote add origin https://github.com/user/project.git # Add remote repo
git add .                                                 # Stage changes
git commit -m "Initial commit"                            # Commit changes
git push -u origin main                                   # Push to GitHub
```

13.6 Collaboration Workflow (Multiple Developers)



13.7 Best Practices

- Always pull the latest changes before pushing.
- Use meaningful commit messages.
- Work on separate branches for features.
- Resolve conflicts carefully when multiple people work on the same files.
- Use .gitignore to avoid pushing unnecessary files.
- Follow a proper branching strategy (e.g. feature branches + pull requests).

Rahul Yadav

14. Git Tags & Release Management

14.1 What is a Git Tag ?

- A tag is a reference to a specific commit, usually used to mark important releases (e.g., v1.0.0).
- Tags do not change like branches. They are fixed to a commit.
- It is mainly used for versioning and release management.

14.2 Types of Tags

Type	Description	Use Case
Lightweight Tag	A simple reference to a commit	Used for temporary or local tags
Annotated Tag	Stores extra information like message, author, date	Recommended for releases (e.g., v1.0.0)

14.3 Common Tag Commands

Command	Description
git tag	List all tags
git tag <tag-name>	Create a lightweight tag
git tag -a <tag-name> -m "message"	Create an annotated tag
git show <tag-name>	Show tag details
git push origin <tag-name>	Push a specific tag to remote
git push origin --tags	Push all tags to remote
git tag -d <tag-name>	Delete a local tag
git push origin --delete <tag-name>	Delete a remote tag

14.4 Tagging Workflow



14.5 Example: Create and Push a Release Tag

```
# Create an annotated tag
git tag -a v1.0.0 -m "Release version 1.0.0"

# Push the tag to remote
git push origin v1.0.0

# Push all tags
git push origin --tags
```

14.6 Best Practices

- Use meaningful tag names (e.g., v1.0.0, v1.1.0).
- Prefer annotated tags for official releases.
- Keep a proper versioning strategy (e.g., Semantic Versioning: vMAJOR.MINOR.PATCH).
- Push tags to remote so the team can access releases.
- Use tags to mark stable production releases.
- Maintain a changelog for each release.

Rahul Yadav

15. Git Stash

15.1 What is Git Stash ?

- Git stash is used to temporarily save your uncommitted changes.
- It helps you switch branches or work on something else without committing.
- Your changes are saved in a special stash area and can be restored later.

15.2 When to Use Git Stash ?

- When you need to quickly switch to another branch.
- When your current work is not complete.
- When you want to pull latest changes without committing.
- When you are working on a bug fix or urgent task.

15.3 How Git Stash Works ?



15.4 Common Stash Commands

Command	Description
git stash	Stash your uncommitted changes
git stash save "message"	Stash with a message (older syntax)
git stash list	List all stashes
git stash show	Show the changes in the latest stash
git stash show -p	Show detailed changes (patch format)
git stash apply	Apply the latest stash (keeps it in the stash list)
git stash apply stash@{n}	Apply a specific stash
git stash pop	Apply the latest stash and remove it from the stash list
git stash drop	Remove the latest stash
git stash drop stash@{n}	Remove a specific stash
git stash clear	Remove all stashes

15.5 Example Workflow

```
# 1. Make some changes
   (modify files without committing)

# 2. Stash the changes
  git stash

# 3. Switch to another branch
  git checkout feature-branch

# 4. Do some work and come back
  git checkout main

# 5. List stashes
  git stash list

# 6. Apply the latest stash
  git stash apply

# 7. Or apply and remove
  git stash pop

# 8. Apply specific stash
  git stash apply stash@{0}
```

15.6 Best Practices

- Use meaningful stash messages (for older versions).
- Apply or pop stashes regularly to keep the list clean.
- Use git stash show to verify changes before applying.
- Avoid keeping too many stashes.
- Use stash for temporary work only, not for long-term storage.
- Commit your changes if the work is complete.

Rahul Yadav

16. Git Rebase

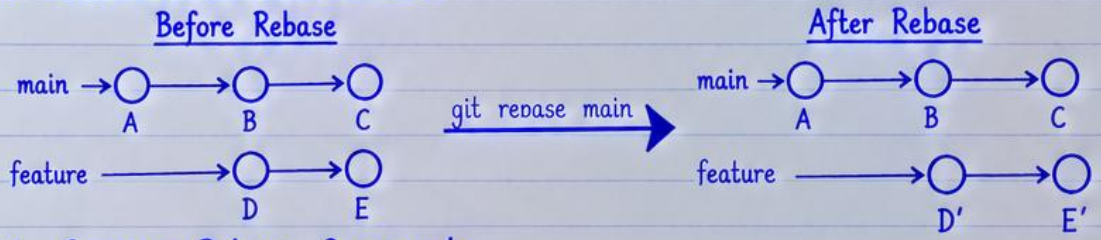
16.1 What is Git Rebase ?

- Git rebase is used to move or combine a series of commits to a new base commit.
- It helps to keep a clean and linear commit history.
- It reapplies your commits on top of another branch.
- Unlike merge, rebase does not create an extra merge commit.

16.2 Rebase vs Merge

Feature	Rebase	Merge
Commit History	Clean and linear	Keeps full history
Creates Merge Commit	No	Yes
Use Case	Keep history clean	Preserve exact history
Best For	Feature branches (before merge)	Collaborative and public branches

16.3 How Git Rebase Works ?



16.4 Common Rebase Commands

Command	Description
git rebase <branch>	Rebase current branch onto another branch
git rebase main	Rebase current branch with latest main
git rebase -i HEAD~n	Interactive rebase (edit/squash commits)
git rebase --continue	Continue rebase after resolving conflicts
git rebase --abort	Cancel the rebase process
git rebase --skip	Skip the current patch
git pull --rebase	Pull latest changes using rebase instead of merge

16.5 Interactive Rebase (Useful Options)

Option	Description
pick	Keep the commit as it is
reword	Change the commit message
edit	Pause to edit the commit
squash	Merge this commit with the previous one
fixup	Like squash but discard this commit message
drop	Remove the commit

16.6 Example: Interactive Rebase

```
git rebase -i HEAD~3
pick a1b2c3d   Commit 1      # Rebase last 3 commits
pick d4e5f6g   Commit 2      # Keep commit 1
squash h7i8j9k Commit 3      # Keep commit 2
                             # Squash commit 3 into commit 2
```

16.7 Best Practices

- Use rebase for your feature branch before creating a pull request.
- Avoid rebasing public or shared branches.
- Use interactive rebase to clean up commit history (squash, reword, drop).
- Always run tests after rebasing.
- Be careful while resolving conflicts.
- Prefer rebase for a clean and readable commit history.

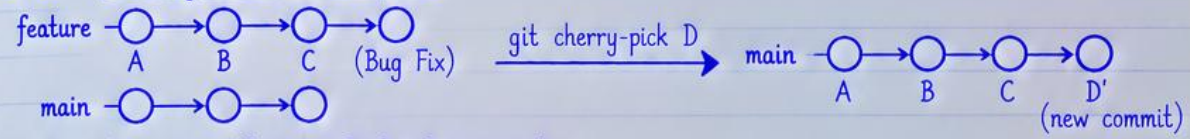
Rahul Yadav

17. Git Cherry-Pick, Reset & Revert

17.1 What is Cherry-Pick ?

- Cherry-pick is used to apply a specific commit from one branch to another.
- It allows you to pick only the changes you need.
- Useful when you want to apply a bug fix or feature from one branch without merging the entire branch.
- It creates a new commit with the same changes.

17.2 Cherry-Pick Workflow



17.3 Common Cherry-Pick Commands

Command	Description
git cherry-pick <commit-hash>	Apply a specific commit to current branch
git cherry-pick <hash>..<hash>	Apply a range of commits
git cherry-pick --no-commit <hash>	Apply changes without creating a commit
git cherry-pick --abort	Cancel cherry-pick if there is a conflict
git cherry-pick --continue	Continue after resolving conflicts

17.4 What is Reset ?

- Reset is used to move the current branch to a specific commit.
- It can be used to undo commits, discard changes, or move to a previous state.
- There are three types of reset: --soft, --mixed (default), --hard.

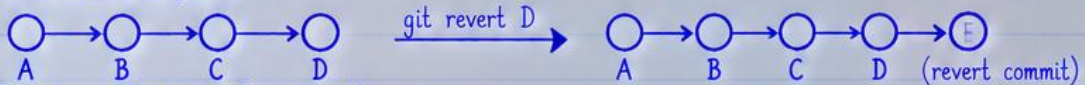
17.5 Types of Reset

Type	Command	Description	Effect
Soft Reset	git reset --soft <hash>	Move to a commit but keep changes in staging area	Keeps changes in staging (working directory unchanged)
Mixed Reset	git reset --mixed <hash>	Move to a commit and keep changes in working directory	Removes from staging (default option)
Hard Reset	git reset --hard <hash>	Move to a commit and discard all changes	Removes changes from both staging and working directory

17.6 What is Revert ?

- Revert is used to create a new commit that undoes the changes of a previous commit.
- It does not delete history and is safe to use on shared branches.
- It keeps the commit history intact.
- Useful when you want to undo a commit in a safe way.

17.7 Revert Workflow



17.8 Common Revert Commands

Command	Description
git revert <commit-hash>	Create a new commit that undoes the changes
git revert <hash1>..<hash2>	Revert a range of commits
git revert --no-commit <hash>	Revert changes without creating a commit
git revert --abort	Cancel the revert process (if there is a conflict)
git revert --continue	Continue after resolving conflicts

17.9 Best Practices

- Use cherry-pick for specific commits only.
- Use reset for local changes (avoid on shared branches).
- Use revert for undoing commits in shared or production branches.
- Always create a backup branch before using reset --hard.
- Read commit messages carefully before cherry-pick or revert.

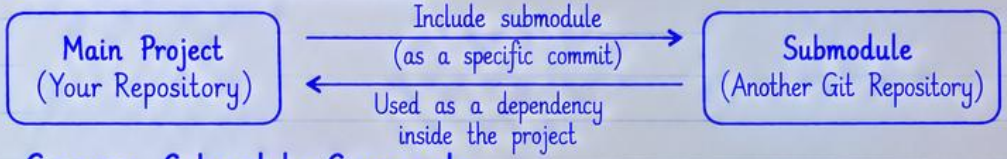
Rahul Yadav

18. Git Submodules, Git Ignore & Diff

18.1 What are Git Submodules ?

- Git submodules allow you to include and manage another Git repository inside your main repository.
- It is useful when your project depends on another project.
- The submodule is a separate repository with its own history.
- It helps in keeping dependencies as a specific version.

18.2 How Git Submodules Work ?



18.3 Common Submodule Commands

Command	Description
git submodule add <repo-url> <path>	Add a new submodule
git submodule init	Initialize submodules
git submodule update	Download submodule content
git submodule update --remote	Update submodule to latest commit
git submodule status	Show submodule status
git submodule deinit <path>	Remove submodule from working directory
git submodule remove <path>	Remove submodule completely

18.4 What is .gitignore ?

- The .gitignore file tells Git which files or folders to ignore.
- Ignored files will not be tracked or committed.
- It is useful to exclude unnecessary files like build files, log files, IDE files, etc.

18.5 Example .gitignore File

```
# Compiled files
*.class                          # Ignore all .class files

# Build folders
/build/                          # Ignore build directory (Gradle)
/target/                         # Ignore build directory (Maven)

# IDE files
.idea/                           # Ignore IntelliJ IDEA folder
*.iml                            # Ignore IntelliJ module files

# Log files
*.log                            # Ignore log files

# OS files
.DS_Store                        # Ignore macOS system files

# Environment files
.env                             # Ignore environment variables file
```

18.6 What is Git Diff ?

- Git diff is used to show the differences between files, commits, branches, or the working directory.
- It helps to see what changes have been made.

18.7 Common Diff Commands

Command	Description
git diff	Show changes in working directory (not staged)
git diff --staged	Show changes in staged files (index)
git diff <commit1> <commit2>	Show differences between two commits
git diff <branch1> <branch2>	Show differences between two branches
git diff --name-only	Show only the list of changed files
git diff --stat	Show summary of changes

18.8 Best Practices

- Use submodules only when necessary.
- Keep .gitignore updated with project-specific files.
- Use git diff regularly to review your changes before committing.
- Avoid committing unnecessary files (build, log, IDE, environment files).
- Maintain clear and consistent ignore rules for team collaboration.

Rahul Yadav

19. Git Workflows & Best Practices

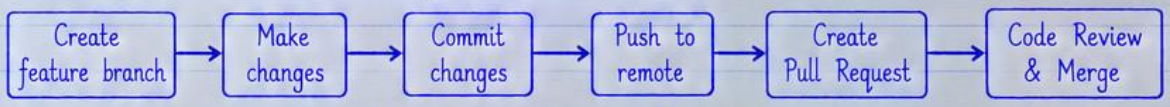
19.1 What is Git Workflow ?

- A Git workflow is a set of rules and practices for using Git in a project.
- It defines how developers create branches, make changes, and merge code.
- A good workflow helps in collaboration, code review and maintaining a clean history.

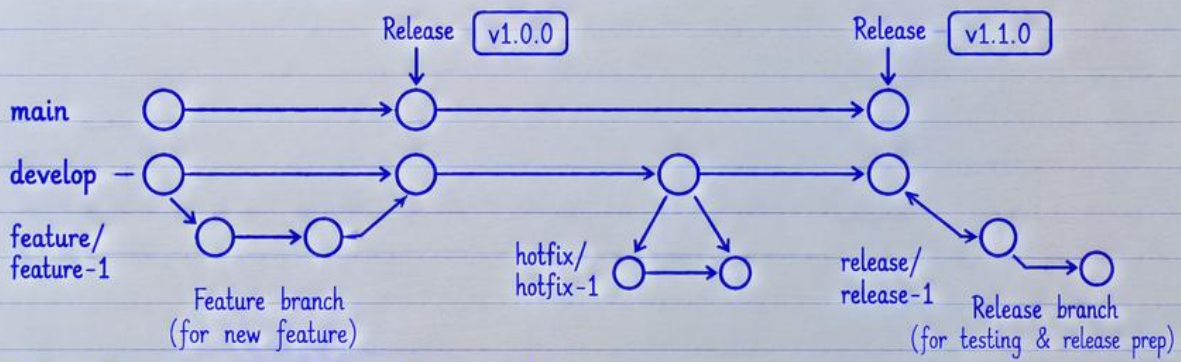
19.2 Popular Git Workflows

Workflow	Description	Use Case
Centralized Workflow	Single main branch, direct commits by all developers	Small teams, simple projects
Feature Branch Workflow	Create a new branch for each feature	Most common, medium to large teams
Gitflow Workflow	Uses multiple branches (main, develop, feature, release, hotfix)	Large projects with scheduled releases
Forking Workflow	Each developer works on a fork and sends pull requests	Open-source projects

19.3 Feature Branch Workflow (Most Common)



19.4 Gitflow Workflow (Branch Structure)



19.5 Common Workflow Commands

Command	Description
git checkout -b feature/feature-name	Create and switch to a new feature branch
git push origin feature/feature-name	Push feature branch to remote
git checkout develop	Switch to develop branch
git merge feature/feature-name	Merge feature branch into develop
git checkout main	Switch to main branch
git merge develop	Merge develop branch into main
git push origin main	Push changes to remote main branch

19.6 Best Practices

- Use feature branches for new changes.
- Keep commits small and meaningful.
- Write clear and descriptive commit messages.
- Use pull requests for code review.
- Keep main branch stable and production-ready.
- Regularly sync with remote repository.
- Use .gitignore to exclude unnecessary files.
- Follow a consistent branching strategy (e.g., Feature Branch or Gitflow).
- Document the workflow and follow team guidelines.

Rahul Yadav